

BoostMySkills — Live Open edX to Current Platform Migration Plan

Source commit: 8e36e6d · **Generated:** 2026-07-22 · **Target DB:** PostgreSQL 16.14 (bms) **Status of this document:** execution plan.

No migration has been executed. No real source export has been supplied; historical learner migration is **externally blocked** (see §18 and docs/migration/migration-readiness.md).

Labelling convention used throughout: each capability is marked **[implemented]**, **[proposed]**, **[externally blocked]**, or **[requires source validation]**. Nothing here claims a completed migration, a password-hash compatibility, or any Production/UAT execution.

1. What "migration" means here — three distinct flows

The current live BoostMySkills website/application is the migration **source**. There are three separate flows that must not be conflated:

- **Flow A — Existing live Open edX content migration.** Course content moves primarily via **OLX** export/import.
- **Flow B — Existing learner / history migration.** Users, enrolments, progress, grades and certificates require **database/API/export data in addition to OLX** — OLX alone does not contain them.
- **Flow C — Later UAT-to-Production content promotion.** Moving natively-authored UAT credentials to Production. This is **not** the one-time live migration; it is an ongoing operational process (see §17).

OLX alone does not contain all users, enrolments, progress, grades or certificates. No migration can be declared complete until actual source exports are supplied and reconciled.

2. Source-data request checklist [requires source validation]

2.1 Course / content source (per live course)

- One OLX .tar.gz per live course; course keys; course display names; start/end dates; organisation/run identifiers; static assets; PDFs; images; videos or video references; problem/MCQ definitions; grading policy; certificate configuration; discussion references; redirects / legacy URLs.

2.2 Open edX LMS relational / export source (depends on deployment version)

- Learner/user identity export; usernames; normalised emails; first/last names; active/deactivated state; account creation date; course enrolments; enrolment modes/status; courseware/unit progress; last access; grades; pass/fail status; certificate records; certificate verification identifiers; completion dates; course/module usage keys.

Potential Open edX entities (equivalents — **exact names must be confirmed against the live deployment, do not assume they exist**): auth_user , student_courseenrollment , courseware_studentmodule , persistent course-grade tables, certificates_generatedcertificate , user-profile tables.

2.3 Credentials / badge sources

- Open edX Credentials service; Parchment / Open Badges data where used; assertion URLs; badge image references; revocation data.

2.4 Business mapping sources

- Project list; organisation names; programme definitions; programme→credential ordering; certificate templates; test/dummy-course decisions; known legacy route mappings.

2.5 Operational source

- Source-system versions; database timezone; export timestamp; source data-freeze timestamp; row counts; checksums; retention constraints.

3. Target data mapping (summary)

The authoritative field-level mapping is in [live-to-current-data-mapping.csv](#) . Summary:

Source	→ Target
Live users	<code>app_users</code> (source id → <code>external_ref</code>)
Open edX course identity	<code>projects</code> / <code>micro_credentials</code>
OLX course structure	<code>credential_versions.content_document</code>
OLX grading / problem configuration	<code>credential_versions.grading_document</code>
OLX source identity	<code>credential_versions.source_metadata</code> + content <code>sourceKey</code> values
Programmes	<code>micro_programmes</code> + <code>programme_credentials</code>
Course enrolments	<code>enrollments</code>
Open edX usage / module progress	<code>unit_progress</code> (only where usage key resolves to an imported unit)
Grades / assessment history	<code>assessment_attempts</code> where detailed attempts exist; otherwise final grade/pass on the enrolment
Certificates	<code>certificates</code>
Legacy verification ids	<code>certificates.verification_code</code> or <code>external_ref</code> per the verified conflict policy
Account-state / deletion	<code>app_users.deactivated_at</code> (and, only where appropriate, <code>account_deletion_requests</code>)

Do NOT migrate: live `platform_settings` maintenance state; `schema_migrations` ; local test-auth identities; local demo/seed data; local filesystem paths.

When detailed attempts do not exist: preserve the final grade/pass on the `enrollment` , do **not** fabricate question-level `assessment_attempts` , and document the limitation.

4. Identity and Clerk migration [requires source validation]

- The source Open edX user id is stored in `app_users.external_ref` .
- The source username is preserved where valid (normalised, CI-unique); email is normalised (trimmed + lower-cased, enforced by CHECK).
- Duplicates and collisions (email/username) are **reported, never silently merged**.
- The **Clerk user id is environment-specific**: local/UAT Clerk ids are **not** copied to Production; `app_users` is mapped to the Clerk user created in the **target** environment.

4.1 Password decision tree [requires source validation]

1. **Import exact source password hash** into Clerk — only if the Open edX hash algorithm and parameters are confirmed and Clerk supports importing that exact hash.
2. **Invitation / password-reset migration** — users set a new password on first access.
3. **First-login claiming flow** — account claimed and verified on first sign-in.
4. **Forced reset for all migrated users.**

Constraints: password migration **cannot be promised** until the Open edX hash algorithm, parameters and Clerk import support are confirmed. Passwords must **never** be exported or stored in plaintext. A password-reset / invitation fallback must be **approved before cutover**.

4.2 Identity reconciliation report [proposed]

Source users; target Clerk users; `app_users` rows; duplicate emails; duplicate usernames; unresolved mappings; disabled/deactivated accounts.

5. Content migration flow (Flow A)

1. Export OLX from the live Open edX. 2. Calculate the archive checksum. 3. Store the original archive privately. 4. **Validate archive safety** — compressed size, expanded size, file count, traversal, absolute paths, Windows drive paths, symlinks, hardlinks, device files, unsafe XML, unsafe HTML.
2. Parse supported OLX structures. 6. Create the Project mapping. 7. Create the stable Micro-Credential identity. 8. Create a **draft** credential revision. 9. Preserve source identifiers as `sourceKey`.
3. Rewrite static/PDF/media references to **provider-neutral object keys**. 11. Separate learner content and grading answers into `content_document` / `grading_document`. 12. Record unsupported XBlocks. 13. Admin review. 14. **Publish only after validation and sign-off**. 15. Preserve the original archive and the import report.

Archive-safety validation is **[implemented]** (`lib/olx/archiveSafety.ts`, `lib/olx/importer.ts`, including DOCTYPE/ENTITY rejection and HTML sanitisation). Unsupported blocks must be **rejected**, represented as a **reviewed placeholder**, or **preserved archive-only**, per an approved decision — never silently discarded.

6. Learner-history migration flow (Flow B) — dependency order

1. Projects → 2. Micro-Credentials → 3. Credential versions → 4. Programmes → 5. Programme memberships → 6. Clerk identities → 7. `app_users` → 8. Enrolments → 9. Unit progress → 2. Grades/Attempts → 11. Certificates.

Stable mapping: Open edX course/module/usage key → `content_document sourceKey` → new stable unit id. Historical progress can only be migrated reliably when the source usage key maps to an imported unit.

Fallback handling: mapped progress is imported; unresolved unit references are reported; course-level completion is preserved where possible; final grade/pass is retained when question-level detail is absent; no synthetic attempts are invented; the certificate remains authoritative where historical details are incomplete.

7. Migration execution phases (runbook)

- **Phase 0 — Approval & access:** owner approval; named source contacts; read-only source access; data-processing approval; Production/UAT handling rules.
- **Phase 1 — Discovery:** source versions; source tables/APIs; course/programme/certificate inventory; user counts; data-quality issues.
- **Phase 2 — Extract:** OLX archives; CSV/SQL/JSON exports; checksums; a **timestamped immutable source bundle**.
- **Phase 3 — Staging & transformation:** encrypted staging; source ids retained; normalised mapping files; dry-run reports; duplicate/conflict reports.
- **Phase 4 — Content dry run:** import all supported courses as **drafts**; compare hierarchy and assets; report unsupported blocks.
- **Phase 5 — Learner-history dry run:** users; enrolments; progress; grades; certificates; reconciliation.
- **Phase 6 — UAT rehearsal:** real or approved anonymised data; business-owner sampling; learner login; history comparison; certificate verification.
- **Phase 7 — Production pre-cutover:** Production backup; source-freeze strategy; delta window; DNS/redirect readiness; Clerk readiness; B2/RDS readiness.
- **Phase 8 — Final cutover:** final source export; delta migration; reconciliation; enable target; redirect traffic.
- **Phase 9 — Hypercare:** monitor failed logins; missing enrolments; progress discrepancies; certificate verification; route errors; rollback criteria.

8. Zero-data-loss and reconciliation

"Zero data loss" is **not** defined as matching total row counts alone. Reconciliation checks:

- **Identity:** user counts; active/inactive counts; unique emails; unique usernames; unresolved users.
- **Content:** course count; section/subsection/unit counts (by type); asset count; unsupported-block count; OLX checksums.
- **Programmes:** programme count; member count; order; required flags.
- **Enrolments:** source vs target count per course; active/completed/withdrawn counts; duplicate detection.

- **Progress:** completed-module counts; completion percentages; unresolved usage keys; sample learner comparisons.
- **Grades:** final percentage; pass/fail; completion date; missing detailed attempts.
- **Certificates:** count; verification code; issued date; status; revocation; public verification result.

Acceptance thresholds: zero missing issued certificates; zero duplicate users caused by migration; zero duplicate enrolments; zero published course without an approved content review; all unresolved records explicitly reported and owner-approved; selected learner samples match source history exactly.

9. Cutover, delta and rollback

Safe cutover: migration rehearsal; immutable source backup; target backup/snapshot; source write-freeze or controlled delta period; final extraction timestamp; **idempotent import/upsert;** final reconciliation; owner sign-off; traffic switch; redirect enablement; monitoring.

Rollback triggers: authentication failure above threshold; missing enrolments; certificate verification failure; major content mismatch; unacceptable unresolved-user count; data corruption; security incident.

Rollback actions: disable target access; restore source routing; restore the target snapshot if required; preserve failed-migration evidence; **do not delete the source system;** communicate to affected users; schedule a corrected rerun.

Zero downtime is not claimed unless the final-delta extraction and traffic switch have actually been rehearsed.

10. Commands and tooling

Implemented (verified to exist in this repository):

Purpose	Command
Apply migrations (idempotent, forward-only)	<code>npm run db:migrate</code>
Seed local/demo data	<code>npm run db:seed / npm run db:seed:ui</code>
Backup (pg_dump custom format)	<code>npm run db:backup</code>
Restore-verify into a scratch DB	<code>npm run db:restore:verify</code>
Reset local DB	<code>npm run db:reset</code>
Learner-import dry run (users)	<code>node --experimental-strip-types scripts/migration/dry-run.mts (--apply to write)</code>
OLX import (admin UI)	<code>/admin/imports → import route (lib/olx/importer.ts)</code>
OLX export (admin)	<code>/admin/credentials/[id]/export (lib/olx/exporter.ts)</code>

The learner dry-run reads `MIGRATION_SOURCE` (default `migration-source/users.json`) and reports `UNAVAILABLE` when no source is present — it never fabricates a "successful" migration.

Required command/script to be implemented [proposed] (interface + I/O described, not built):

- *Content bulk-import orchestrator* — input: a directory of OLX archives + a course→project map; output: draft credential revisions + an import report (per-course hierarchy, asset and unsupported-block counts).
- *Enrolment/progress/grade importer* — input: normalised CSV/JSON exports keyed by `external_ref` /usage keys; output: `enrollments / unit_progress / assessment_attempts` upserts
 - an unresolved-key report.
- *Certificate importer* — input: source certificate records; output: `certificates` upserts + a verification-code conflict report.
- *Reconciliation reporter* — input: source counts + target counts; output: the §8 reconciliation tables with pass/fail against the acceptance thresholds.

10.1 Intended staging layout (git-ignored; never contains committed personal data)

```
.data/migration/<run-id>/
source/      # immutable raw exports + OLX archives + checksums
normalized/  # mapping files keyed by external_ref / usage key
reports/     # dry-run + reconciliation reports
rejected/    # unresolved / rejected records
manifests/   # per-run manifest + checksums
```

`.data/` is already git-ignored. No personal data is ever committed.

11. UAT-to-Production OLX promotion (Flow C — separate from the live migration)

For a natively-authored UAT credential:

1. Freeze/publish the approved UAT draft. 2. Export OLX. 3. Generate checksum + manifest. 4. Transfer through an approved private channel. 5. Admin imports into Production. 6. Import remains **draft**.
2. Validate structure, assets, grading and metadata. 8. Compare manifest/checksum. 9. Admin publishes in Production. 10. Smoke-test catalogue/player. 11. Record promotion evidence.

Do NOT migrate in Flow C: UAT learners; UAT enrolments; UAT progress; UAT attempts; UAT certificates; UAT Clerk ids; UAT platform settings.

Local export/re-import proves only the **mechanism**. A real UAT-to-Production promotion must still be executed and verified before that acceptance criterion is complete.

12. Open decisions and blockers

Decision	Owner	Deadline	Consequence if unresolved
Access to live Open edX database/exports	Client / infra	before Phase 1	Flow B cannot start
Confirmed Open edX version	Client	before Phase 1	source table/API shapes unknown
Clerk password-hash import feasibility	Project owner + Clerk	before cutover	password strategy defaults to reset/invitation
Duplicate-user policy	Project owner	before Phase 3	identity reconciliation blocked
Test/dummy-course policy	Business owner	before Phase 4	junk content risks import
Programme source / mapping	Business owner	before Phase 3	programmes cannot be assembled
Parchment / Open Badges handling	Project owner	before Phase 5	badge data unmapped
Certificate verification-code conflict policy	Project owner	before Phase 5	legacy verification links may break
Static PDF/media rewrite rules	Infra	before Phase 4	asset references unresolved
Unsupported XBlock policy	Business owner	before Phase 4	content silently lost (not allowed)
Discussions handling	Business owner	before Phase 4	discussion data unmapped
Legacy redirects inventory	Client	before Phase 7	old URLs 404 after cutover
Production→UAT personal-data policy	Data owner / legal	before Phase 6	rehearsal data handling blocked
Anonymisation approach	Data owner	before Phase 6	UAT rehearsal blocked
Retention constraints	Legal	before Phase 2	export scope undefined
Account deletion vs certificate verification	Project owner	before cutover	deactivation/verification conflict
Migration outage window	Business owner	before Phase 7	cutover cannot be scheduled

Decision	Owner	Deadline	Consequence if unresolved
Final data-freeze owner	Business owner	before Phase 8	delta window undefined

13. Summary

Flow A (content via OLX) has an **implemented, safety-hardened import mechanism** and a clear draft→review→publish path. Flow B (learners/history) has a **defined plan and a dry-run scaffold** but is **externally blocked** on real source exports and the Clerk identity/password decision. Flow C (UAT→Production promotion) has a defined procedure whose mechanism is locally provable but whose real execution remains outstanding. **No migration has been executed and none is claimed.**